

LV01 클래스와 구조체 차이, OOP

<https://youtu.be/Vm2fNaEc1Lo?feature=shared>

<https://youtu.be/WtMHY0KS9qI?feature=shared>

C++ 클래스와 객체지향 프로그래밍 기초

클래스(Class)란?

클래스의 정의와 개념

C++에서 클래스(class)란 **구조체의 상위 호환**으로 이해할 수 있습니다. C++의 구조체는 멤버로 함수를 포함할 수 있기에, C언어의 구조체보다 좀 더 확장된 의미를 가집니다.

C++에서 이러한 구조체와 클래스의 차이는 **기본 접근 제어의 차이일 뿐**, 나머지는 거의 같습니다.

구조체 vs 클래스:

- **struct**: 기본 접근 제어가 `public`
- **class**: 기본 접근 제어가 `private`

C++에서는 이러한 클래스를 가지고 **객체 지향 프로그램을 작성**할 수 있습니다.

객체지향 프로그래밍(OOP, Object-Oriented Programming)

객체지향 프로그래밍의 핵심 개념:

- 모든 데이터를 ****객체(object)****로 취급
- 객체가 바로 **프로그래밍의 중심**이 됨
- 객체(object)란 **실생활에서 우리가 인식할 수 있는 사물**로 이해 가능

객체의 구성 요소:

- **상태(state)**: 객체가 가지고 있는 데이터 (멤버 변수)

- **행동(behavior)**: 객체가 수행할 수 있는 동작 (멤버 함수)

이러한 객체의 상태(state)와 행동(behavior)을 구체화하는 형태의 프로그래밍이 바로 **객체지향 프로그래밍**입니다.

클래스와 객체의 관계:

- **클래스(class)**: 객체를 만들어 내기 위한 틀과 같은 개념
- **객체(object)**: 클래스를 바탕으로 실제로 생성된 **인스턴스**

구조체를 이용한 전통적인 방법

일반 함수를 이용한 구조체 처리

일반적으로 함수를 이용해서 인스턴스를 넘겨줘서 출력을 해줬을 것입니다.

```
#include <iostream>
using namespace std;

struct Cat
{
    int age;
    int weight;
    char name[20];
};

void PrintCat(const Cat* cat)
{
    cout << "Cat's name: " << (*cat).name << endl;
    cout << "Cat's age: " << (*cat).age << endl;
    cout << "Cat's weight: " << (*cat).weight << endl;
}

int main()
{
    Cat yamyami;
    yamyami.age = 7;
    yamyami.weight = 4;

    // 문자열 복사 (배열에는 직접 대입 불가)
```

```

const char name[20] = "yamyami";
for (int i = 0; i < 20; i++)
{
    yamyami.name[i] = name[i];
}

Cat geegee;
geegee.age = 3;
geegee.weight = 5;
const char geegee_name[20] = "geegee";
for (int i = 0; i < 20; i++)
{
    geegee.name[i] = geegee_name[i];
}

PrintCat(&yamyami);
PrintCat(&geegee);

return 0;
}

```

전통적인 방법의 단점

반복적인 작업의 불편함:

- 함수 호출 시마다 해당 클래스 인스턴스(객체)를 인자로 넣어줘야 하기 때문에 귀찮습니다
- 데이터와 그 데이터를 처리하는 함수가 분리되어 있어 관리가 어려움
- 실수로 잘못된 객체를 전달할 위험성

멤버 함수 (Member Function)

클래스 내부에 함수 포함

그래서 함수를 클래스 또는 구조체 안에 넣어버려서 멤버 변수처럼 멤버함수를 만들었습니다.

멤버함수의 동작 원리

C++ 멤버함수는 내부적으로 어떻게 동작할까요? 실제로는 C 언어와 비슷하게 동작합니다. 컴파일러가 자동으로 `this` 포인터를 첫 번째 매개변수로 전달합니다.

C 언어 스타일 (컴파일러가 실제로 하는 일):

```
// 컴파일러가 내부적으로 변환하는 방식
struct Cat
{
    int age;
    int weight;
    char name[20];
};

// 멤버함수는 실제로 이렇게 변환됨
void Cat_PrintCat(Cat* this)
{
    cout << "Cat's name: " << this->name << endl;
    cout << "Cat's age: " << this->age << endl;
    cout << "Cat's weight: " << this->weight << endl;
}

void Cat_SetAge(Cat* this, int age)
{
    this->age = age;
}

// 함수 호출도 이렇게 변환됨
int main()
{
    Cat yamyami;

    // yamyami.PrintCat(); → 실제로는 이렇게 변환
    Cat_PrintCat(&yamyami);

    // yamyami.SetAge(7); → 실제로는 이렇게 변환
    Cat_SetAge(&yamyami, 7);
}
```

```
    return 0;
}
```

C++ 클래스 스타일 (우리가 작성하는 방식):

```
#include <iostream>
using namespace std;

class Cat
{
public:
    void PrintCat(/*Cat* this*/)
    {
        // 컴파일러가 자동으로 this→를 붙여줌
        cout << "Cat's name: " << name << endl;    // this→name
        cout << "Cat's age: " << age << endl;      // this→age
        cout << "Cat's weight: " << weight << endl; // this→weight
    }

    void SetAge(int age)
    {
        // 매개변수와 멤버변수 이름이 같을 때는 this→ 필수
        this→age = age; // this→age = 매개변수 age
    }

    int age;
    int weight;
    char name[20];
};

int main()
{
    Cat yamyami;
    yamyami.age = 7;
    yamyami.weight = 4;

    const char name[20] = "yamyami";
    for (int i = 0; i < 20; i++)
```

```

{
    yamyami.name[i] = name[i];
}

Cat geegee;
geegee.age = 3;
geegee.weight = 5;
const char geegee_name[20] = "geegee";
for (int i = 0; i < 20; i++)
{
    geegee.name[i] = geegee_name[i];
}

// 멤버 함수 호출 - 훨씬 간편해짐!
yamyami.PrintCat(); // 내부적으로 Cat_PrintCat(&yamyami) 호출
geegee.PrintCat(); // 내부적으로 Cat_PrintCat(&geegee) 호출

return 0;
}

```

this 포인터의 이해

this 포인터란?

this 포인터는 현재 객체의 주소를 가리키는 특별한 포인터입니다. 모든 멤버함수는 암시적으로 this 포인터를 첫 번째 매개변수로 받습니다.

this 포인터의 특징:

- **자동 전달:** 컴파일러가 자동으로 전달
- **현재 객체:** 멤버함수를 호출한 객체의 주소
- **암시적 사용:** 보통은 생략 가능
- **명시적 사용:** 필요시 직접 사용 가능

this 포인터의 타입

this 포인터의 타입은 상황에 따라 결정됩니다:

```

class MyClass
{
public:
    void normalFunction()    // this의 타입: MyClass*
    {
        // this를 통해 멤버 변수 수정 가능
    }

    void constFunction() const // this의 타입: const MyClass*
    {
        // this를 통해 멤버 변수 수정 불가
        // value = 10; // 오류!
    }

private:
    int value;
};

```

멤버 함수의 장점

코드 간소화:

- `PrintCat(&yamyami)` → `yamyami.PrintCat()`
- 객체 자신의 멤버 변수에 **자동으로 접근** 가능
- 별도의 매개변수 전달 불필요

객체지향적 설계:

- 데이터(멤버 변수)와 기능(멤버 함수)이 **하나의 클래스로 묶임**
- 관련된 기능들이 한 곳에 모여 **유지보수성 향상**

접근 제어와 캡슐화

Private 접근 제어의 필요성

하지만 대게 또 **멤버변수는 접근제한 private** 가려놓는게 좋습니다. 왜? 데이터 임의접근해서 값을 바꾸는 행위는 프로그램에 큰 문제를 야기시킬 수 있기 때문에 대게 감춰둡니다. 접근하지 못하도록 합니다.

그래서 우리는 값을 바꾸거나 가져오는 행위를 가능하게 하기 위해서 **get, set** 함수를 만들어서 사용합니다.

완전한 캡슐화 구현

```
#include <iostream>
using namespace std;

// 객체 지향 프로그래밍 완성형
class Cat
{
public:
    // 정보 출력 함수
    void PrintCat()
    {
        cout << "Cat's name: " << mName << endl;
        cout << "Cat's age: " << mAge << endl;
        cout << "Cat's weight: " << mWeight << endl;
    }

    // Setter 함수들 (값 설정)
    void SetAge(int age)
    {
        if (age >= 0 && age <= 30) // 유효성 검사
        {
            mAge = age;
        }
        else
        {
            cout << "Invalid age!" << endl;
        }
    }

    void SetWeight(int weight)
    {
        if (weight > 0 && weight <= 50) // 유효성 검사
        {
            mWeight = weight;
        }
    }
};
```



```

    }
    else
    {
        cout << "Invalid weight!" << endl;
    }
}

void SetName(const char* name)
{
    // 안전한 문자열 복사
    for (int i = 0; i < 19; i++) // 19까지만 복사 (null 문자 공간 확보)
    {
        if (name[i] == '\0') break; // 문자열 끝에 도달
        mName[i] = name[i];
    }
    mName[19] = '\0'; // null 문자 보장
}

// Getter 함수들 (값 조회)
int GetAge() const { return mAge; }
int GetWeight() const { return mWeight; }
const char* GetName() const { return mName; }

private:
    int mAge;
    int mWeight;
    char mName[20];
};

int main()
{
    Cat yamyami;

    // 직접 접근 불가 (컴파일 오류)
    // yamyami.mAge = 7; // Error!

    // Setter 함수를 통한 안전한 접근
    yamyami.SetAge(7);

```

```

yamyami.SetWeight(4);
yamyami.SetName("yamyami");

Cat geegee;
geegee.SetAge(3);
geegee.SetWeight(5);
geegee.SetName("geegee");

// 정보 출력
yamyami.PrintCat();
geegee.PrintCat();

// Getter 함수 사용 예시
cout << "Yamyami's age: " << yamyami.GetAge() << endl;

return 0;
}

```

접근 제어 지시자

접근 제어의 종류

접근 제어	설명	접근 범위
public	어디서든 접근 가능	클래스 외부에서도 접근
private	같은 클래스 내부에서만 접근	클래스 내부 멤버 함수만
protected	상속받은 클래스에서 접근 가능	상속 관계에서 사용

멤버 변수 명명 규칙

일반적인 명명 규칙:

- **mAge, mWeight, mName**: 'm' 접두사로 멤버 변수 표시
- **_age, _weight, _name**: 언더스코어 접두사 사용
- **age_, weight_, name_**: 언더스코어 접미사 사용

```

class Example
{
private:

```

```
int mAge;    // Microsoft 스타일
int _weight; // 일반적인 스타일
int height_; // Google 스타일
char name[20]; // 접두사 없이 사용
};
```

캡슐화의 장점

데이터 보호와 유효성 검사

캡슐화(Encapsulation)의 핵심 장점:

1. **데이터 보호**: 외부에서 직접 수정 불가
2. **유효성 검사**: Setter에서 올바른 값인지 확인
3. **일관성 유지**: 데이터 변경 시 추가 작업 수행 가능
4. **인터페이스 안정성**: 내부 구현 변경해도 외부 코드 영향 없음

실제 활용 예시

```
class BankAccount
{
public:
    void Deposit(int amount)
    {
        if (amount > 0)
        {
            mBalance += amount;
            cout << amount << "원이 입금되었습니다." << endl;
        }
        else
        {
            cout << "입금액은 0보다 커야 합니다." << endl;
        }
    }

    void Withdraw(int amount)
    {
        if (amount > 0 && amount <= mBalance)
```

```

    {
        mBalance -= amount;
        cout << amount << "원이 출금되었습니다." << endl;
    }
    else
    {
        cout << "출금할 수 없습니다. 잔액을 확인하세요." << endl;
    }
}

int GetBalance() const { return mBalance; }

private:
    int mBalance = 0; // 직접 접근 금지!
};

```

캡슐화가 없다면:

- `account.mBalance = -1000;` (음수 잔액 가능!)
- `account.mBalance = 999999999;` (불법적인 잔액 조작!)

캡슐화로 보호받는 경우:

- 입출금은 반드시 `Deposit()`, `Withdraw()` 함수를 통해서만
- 잔액 조회는 `GetBalance()` 로만 가능
- 모든 거래에 유효성 검사 적용

객체지향 프로그래밍의 장점

코드 재사용성과 유지보수성

전통적인 절차지향 vs 객체지향:

```

// 절차지향 (불편함)
PrintCat(&cat1);
PrintCat(&cat2);
SetCatAge(&cat1, 5);
SetCatWeight(&cat1, 3);

// 객체지향 (간편함)

```

```
cat1.PrintCat();
cat2.PrintCat();
cat1.SetAge(5);
cat1.SetWeight(3);
```

객체지향의 핵심 장점:

- **직관적:** 실세계의 사물을 그대로 모델링
- **재사용성:** 한 번 만든 클래스를 여러 번 사용
- **유지보수성:** 관련 기능이 한 곳에 모여 있음
- **확장성:** 상속을 통한 기능 확장 가능
- **안전성:** 캡슐화를 통한 데이터 보호

"강의는 많은데, 내 실력은 왜 그대로일까?"

혼자서 공부하다 보면

이런 생각 들지 않으셨나요?

- 강의는 다 듣고도 **직접 코드**는 못 짜겠고,
- 복습할 땐 **어디서부터 다시 시작해야 할지** 막막하고,
- 질문하려 해도 **물어볼 사람이** 없고,
- 유튜브 영상도 **정답만 보고 따라 치는** 느낌...

그렇다면 지금이 바로

"나만을 위한 코칭"이 필요한 순간입니다.

당신도 할 수 있습니다.

지금 멤버십을 넘어, 코칭에 도전해보세요.

수많은 수강생들이 **얌얌코딩 코칭**으로 넥슨, 크래프톤, NC 등 입사에 성공했습니다.

 **프리미엄 코칭 안내 바로가기**

 또는 카톡 오픈채팅: **얌얌코딩 상담방**

지금도 코딩을 '따라 치기만' 하고 계신가요?

이젠 **혼자 설계하고, 스스로 코딩하는 법**을 배워야 할 때입니다.

얌얌코딩이 옆에서 함께하겠습니다. 💪

▼ oopy:hide

연습문제



문제 1번

세 문장을 2차 배열(3 x 10)에 입력 받아주세요

각 문장의 맨 마지막 글자들을 출력하시면 됩니다

각 문장 마지막 글자를 찾는 함수를 만들어서 풀어주세요.

함수 원형 : `char findChar(char* str)` 과 같은 형태로 작성하여 풀어주시면 됩니다.

M	I	N	C	O	D	I	N	G	₩0
A	P	P	L	E	₩0				
S	K	T	O	W	N	₩0			

출력결과 : GEN

입력 예제

MINCODING

APPLE

SKTOWN

출력 결과

GEN



문제 2번

아래 배열을 하드코딩 하주세요.

A	B	K	T
K	F	C	F
B	B	Q	Q
T	P	Z	F

문자 2개를 입력 받아주세요.

그리고 입력받은 문자들이 배열에 몇개 존재하는지 찾는 함수를 만들어 출력 하주세요.

함수 원형 : `int findCharCount(char str[4][4])` 와 같은 형태입니다.

만약 B, F를 입력 받았다면,

B : 3개, F : 3개 총 6개이므로 6을 출력하면 됩니다.

입력 예제

B F

출력 결과

6

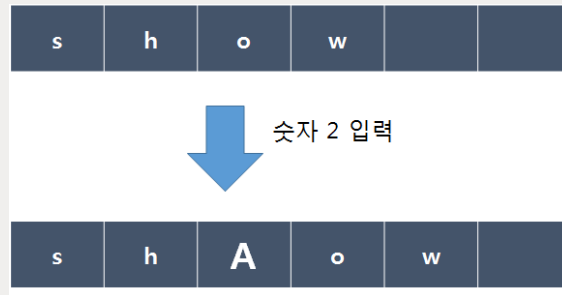


문제 3번

한 문장과 숫자 하나를 입력 받으세요 (입력 문장은 최대 5글자)

그 숫자에 해당하는 배열 index에다가 문자 "A"를 추가한 후 출력 하는 함수를 만들어

주세요.



입력 예제

show

2

출력 결과

shAow



문제 4번

크기가 4인 배열 A B 두개를 만들고,

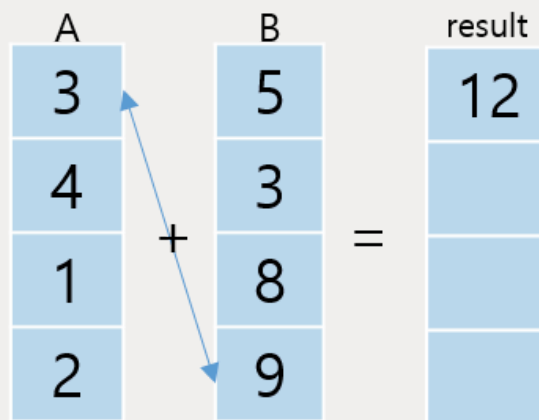
숫자 8개를 A배열과 B배열에 입력받아주세요

1중 for문을 돌려

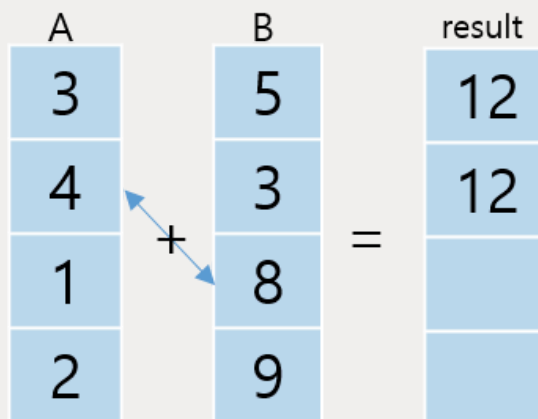
아래의 규칙대로 result 배열을 완성시켜주세요

완성된 result 배열을 출력하면 됩니다.

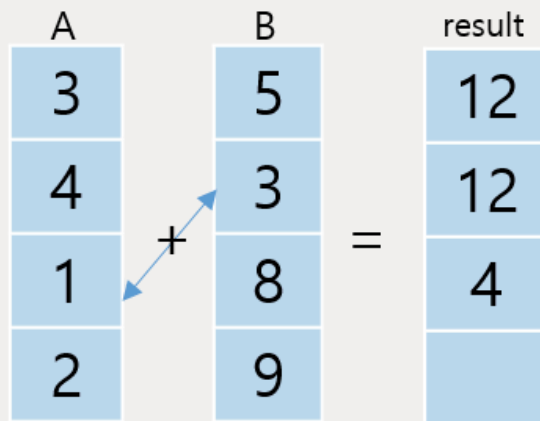
과정1



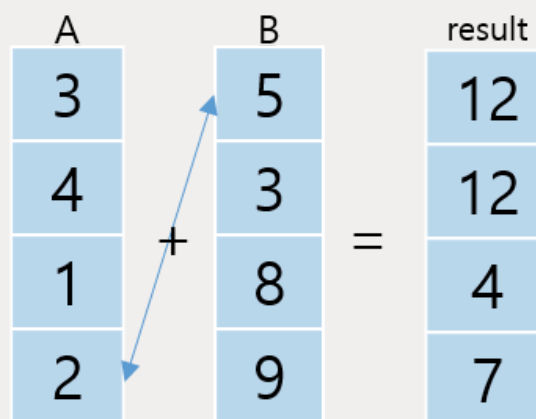
과정2



과정3



과정4



입력 예제

3 4 1 2

5 3 8 9

출력 결과

12 12 4 7

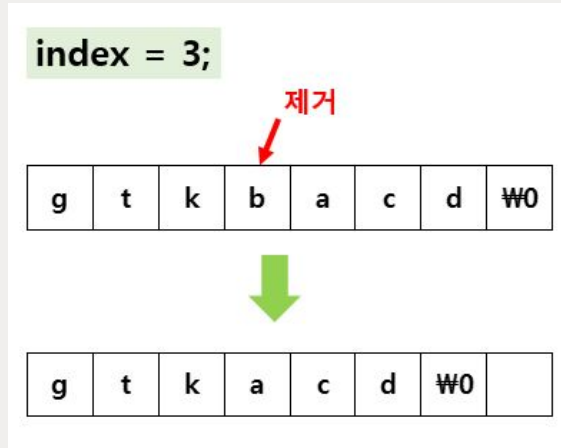


문제 5번

문장 하나를 입력받고 제거할 index를 입력 받으세요.

그리고 새로 만든 함수에 해당 문장과 index를 전달인자로 전달해주세요.

해당하는 index의 글자를 제거하고 출력하는 함수를 만드세요.



입력 예제

gtkbacd

3

출력 결과

gtkacd



문제 6번

세 문장을 입력받고 이 문장에 M 이라는 글자가 존재하는지 찾아 출력 해 주세요.

(각 문장당 최대 10글자)

isExist 함수를 이용하여 찾아 주세요.

M을 찾으셨다면 **M이 존재합니다**

M을 찾지 못하였다면 **M이 존재하지 않습니다**

를 출력 해 주세요.

입력 예제

CODING

MBC

WORLD

출력 결과

M이 존재합니다



문제 7번

아래 배열을 하드코딩 해 주세요

M	T	K	C
---	---	---	---

그리고 찾을 문자를 하나 입력받아주세요

Flag기법을 쓰지않고, **isExist**함수를 써서

입력받은 문자가 위 배열에 존재하는지 출력 해 주세요

"발견" 또는 "미발견"을 출력하시면 됩니다

입력 예제

T

출력 결과

발견



문제 8번

아래 배열을 하드코딩 하고, 문자 2개를 입력받아주세요

G	K	T
P	A	C

입력받은 문자 2개가 모두 배열안에 존재한다면 "대발견" 출력,

문자 둘 중 하나만 발견된다면 "중발견" 출력,

아무 문자도 없다면 "미발견" 출력 해 주세요

isExist 함수를 써서 문제를 풀어주세요.

입력 예제

G A

출력 결과

대발견



문제 9번

숫자 6개를 배열에 입력받아 주세요.

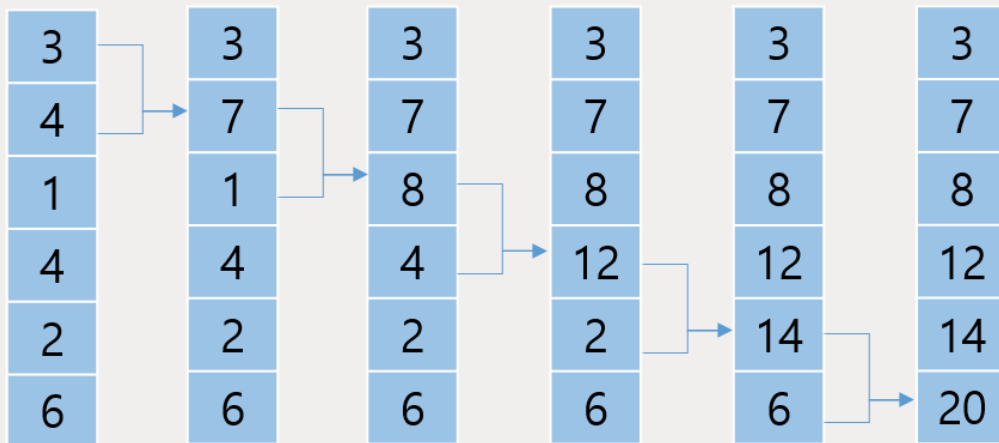
1차원 배열에 1중 for문을 다음과 같은 규칙 돌려주세요.

반복적으로 누적합을 아래와 같이 구하면 됩니다.

누적합을 구하는 연산을 함수로 묶어서 구현해주세요.

누적합 구하는 함수를 5번 반복실행하여

최종적으로 완성된 배열을 출력 해 주세요.



입력 예제

3 4 1 4 2 6

출력 결과

3 7 8 12 14 20



문제 9번

자동차 클래스를 설계하고 객체를 생성하여 화면에 출력해보세요.

자동차 클래스는 4개의 Wheel(바퀴) 객체를 가지고 있고 속도와 연료를 가지고 있습니다.

적절한 데이터 타입을 활용하여 다음과 같은 클래스를 설계해보세요.

모든 멤버변수 값들은 private으로 선언해야 합니다.

객체는 다음과 같이 2개 만들어주세요. ferai, avante 객체 2개를 생성하고 각각 임의의 맞게 값을 대입해주세요. 그러기 위해선 멤버함수를 따로 선언하여 해당

객체의 값을 세팅해줄수 있는 함수를 만들어야 합니다.

그리고 PrintData라는 멤버함수를 만들어서 각각의 값을 콘솔창에 출력 가능하게 만들어주세요.

출력결과

feraril

speed : 200km

fuel : 100l

wheel[0] : size 5inch radius 20cm

wheel[1] : size 5inch radius 20cm

wheel[2] : size 5inch radius 20cm

wheel[3] : size 5inch radius 20cm

avante

speed : 100km

fuel : 50l

wheel[0] : size 3inch radius 20cm

wheel[1] : size 3inch radius 20cm

wheel[2] : size 3inch radius 20cm

wheel[3] : size 3inch radius 20cm

복습문제



문제 1번

숫자 3개를 변수 a, b, c에 입력 받으세요.

출력함수를 만들어 전달인자로 넘겨주세요.

함수 안에서는 a ~ b까지 숫자를 c번 반복해서 출력 해 주세요

입력 예제

2 5 3

출력 결과

2 3 4 5

2 3 4 5

2 3 4 5



문제 2번

아래와 같이 'A'에서부터 알파벳 순서대로 2차 배열에 값을 채워주세요
(2중 for문을 이용해서 값을 채워주세요)

R	L	F
Q	K	E
P	J	D
O	I	C
N	H	B
M	G	A



그리고 좌표를 입력 받고, 좌표에 해당하는 값을 출력 해 주세요

[힌트] 아래와 같이 for문을 돌리면 됩니다.

아래와 같이 for를 돌리면 되는 이유를 생각해 보세요

```
for (x=2; x>=0; x--)
```

```
for (y=5; y>=0; y--)
```

입력 예제

0 2

출력 결과

F



문제 3번

숫자 6개를 입력받고, 2x3배열에 입력받은 숫자로 값을 채워주세요

만약 1, 3, 5, 2, 6, -5를 입력 받았다면, 아래와 같이 값을 채우면 됩니다.

그리고

max의 좌표를 구해주는 함수와

min의 좌표를 구해주는 함수를 각각 만들고,

Max와 Min의 좌표를 출력 해주세요.

1	3	5
2	6	-5

[힌트1]

```
void abc(int* p)
```

```
{
```

```
    *p = 100;
```

```
}
```

```
int main()
```

```
{
```

```
    int d;
```

```
    abc(&d);
```

```
    cout << d; //100이 출력 됩니다.
```

```
    return 0;
```

```
}
```

[힌트2]

```
void getMax(int* dy, int* dx);
```

```
void getMin(int* dy, int* dx);
```

이렇게 두 함수를 만들면 되고,

main 함수에서는

```
getMax(&maxY, &maxX);
```

```
getMin(&minY, &minX);
```

이렇게 함수 호출하면 max좌표와 min좌표를 전달받을 수 있습니다.

입력 예제

1 3 5 2 6 -5

출력 결과

(1,1)

(1,2)



문제 4번

숫자 2개를 먼저 index 0과 index 1에 입력 받으세요

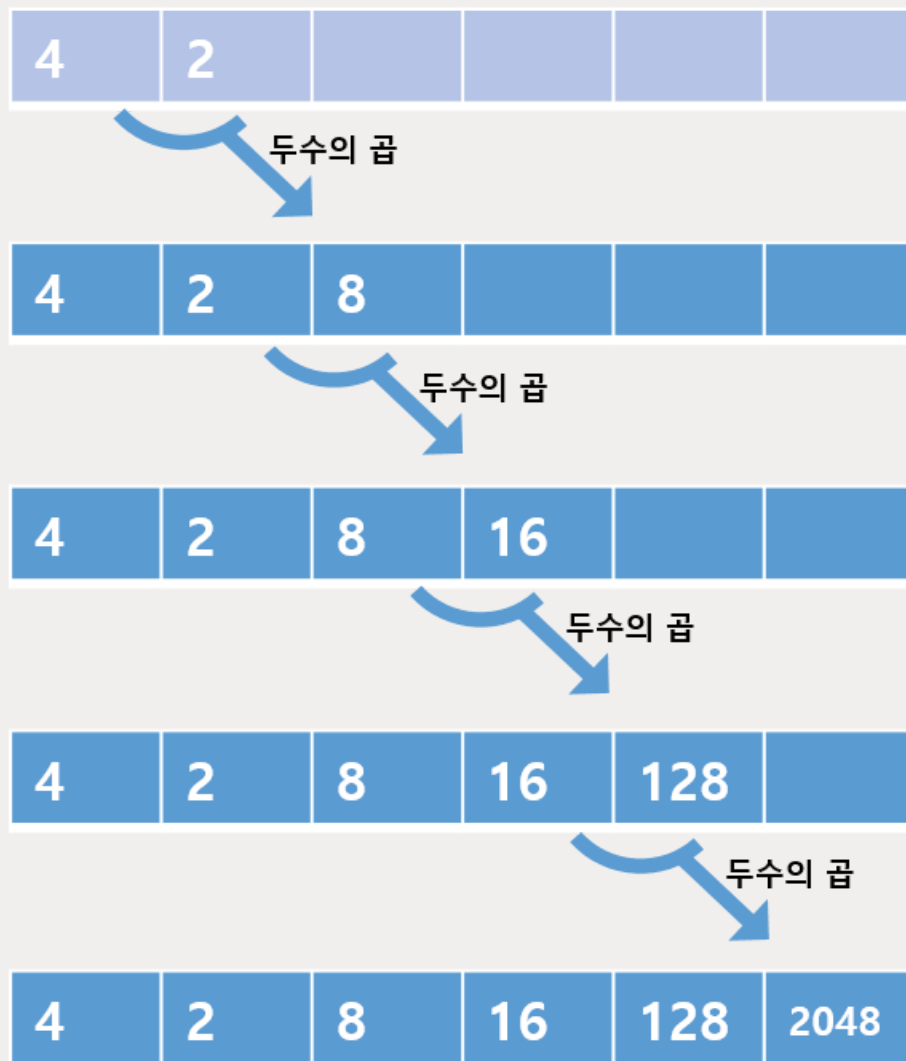
그리고 아래 규칙처럼 총 6칸에 숫자를 모두 채운 후

두 수의 곱을 구하는 함수를 만들어주세요.

해당 함수에 배열의 2개의 인자를 반복적으로 넣어 아래 그림처럼

최종적으로 나오는 숫자를 출력 해 주세요

4 2를 입력했다면



입력 예제

4 2

출력 결과

2048

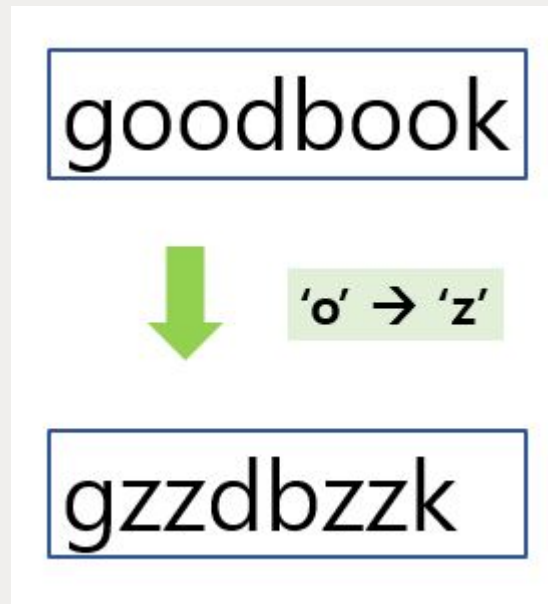


문제 5번

문장 하나를 입력받으세요

그리고, 각각의 변수 a, b에 문자를 입력 받아주세요

문장에 입력받은 a의 문자값이 있다면, b의 문자값으로 바꾸어주는 함수를 만들어 주세요.



입력 예제

goodbook

o

z

출력 결과

gzzdbzzk



문제 6번

대문자로 구성된 한 문장을 입력 받으세요. (최대 10글자)
한 문장에서 사전순으로 가장 뒤에 나오는 글자의 index와
가장 앞에 나오는 글자의 index를 찾아 출력 해 주세요.
max, min 함수를 이용해 문제를 풀어주세요.
(단, 같은 글자는 입력하지 않습니다.)

예로들어 FEBIZ를 입력한다면

max에 해당하는 알파벳은 Z 이고, **4번 index**에 있습니다.

min에 해당하는 알파벳은 B 이고, **2번 index**에 있습니다.

출력결과는

4

2

입력 예제

ABCDE

출력 결과

4

0



문제 7번

7×4 배열을 만들고, 2중 for문을 돌려 아래 그림과 같이 값을 채워주세요.

0번 Line	1	2	3	4
1번 Line	5	6	7	8
2번 Line	9	10	11	12
3번 Line	13	14	15	16
4번 Line	17	18	19	20
5번 Line	21	22	23	24
6번 Line	25	26	27	28

그리고 숫자 3개를 입력 받습니다.

각 숫자에 해당하는 Line을 0으로 바꾸어주세요.

만약 1, 2, 5를 입력 받았다면

1번Line, 2번Line, 5번Line을 0으로 채워주는 함수를 만들어 주고
연산한후 출력하면 됩니다.

0번 Line	1	2	3	4
1번 Line	0	0	0	0
2번 Line	0	0	0	0
3번 Line	13	14	15	16
4번 Line	17	18	19	20
5번 Line	0	0	0	0
6번 Line	25	26	27	28

입력 예제

1 2 5

출력 결과

1 2 3 4

0 0 0 0

0 0 0 0

13 14 15 16

17 18 19 20

0 0 0 0

25 26 27 28



문제 8번

두 문장을 아래와 같이 하드코딩 해 주세요.

A	7	9	T	K	Q
M	I	N	C	O	D

두 글자를 입력받고 두 글자의 존재 여부를 출력 해 주세요.

isExist 함수를 사용하여 문제를 풀어 주세요.

A와 Z를 입력하였다면

A : 존재

Z : 없음

을 출력 해 주세요.

입력 예제

A Z

출력 결과

A : 존재

Z : 없음

Level16.5 사각형 출력하기 [난이도 : 4]

문제 9번

숫자 2개와 문자 1개를 입력 받으세요

그리고 아래 예제에서 규칙을 찾아 출력 해 주세요

(3중 for문을 써서 문제를 풀어주세요)

ex) 만약 2 3 A 를 입력 받았다면

AAA

AAA

AAA

AAA

이렇게 출력하시면 됩니다

ex) 만약 3 5 B 를 입력 받았다면

BBBBB

BBBBB

BBBBB

BBBBB

BBBBB

BBBBB

이렇게 출력하시면 됩니다

[힌트] 2중 for문을 아래와 같이 감싸면

3중 for문으로 해결할 수 있습니다.

```
for (z = 0; z<2; z++)
```

```
{
```

```
// 2중 for문으로 사각형 출력
```

```
}
```

입력 예제

2 4 R

출력 결과

RRRR

RRRR

RRRR

RRRR



문제 10번

Player 클래스는 Weapon 무기 객체를 가지고 있고 hp를 가지고 있습니다. Weapon은 문자열(char 배열)로 구성되어 있습니다.

힌트

```
struct Name
{
    char firstName[20];
    char lastName[20];
};
```

```
struct Point
{
    int x;
    int y;
};
```

```
struct Player
{
    Name name;
    Point position;
};
```

```
class Weapon
{
    };
```

```
class Player
{
private:
    Weapon myWeapon;
};
```

```
int main()
{

    return 0;
}
```

적절한 데이터 타입을 활용하여 다음과 같은 클래스를 설계해보세요.

모든 멤버변수 값들은 private으로 선언해야 합니다.

객체는 다음과 같이 2개 만들어주세요. warrior, archer 객체 2개를 생성하고

각각 무기에 sword bow값으로 세팅해주세요. 그러기 위해선 멤버함수를 따로 선언하여

객체의 값을 세팅해줄수 있는 함수를 만들어야 합니다.

그리고 PrintInfo 멤버함수를 만들어서 각각의 값을 콘솔창에 출력 가능하게 만들어주세요.

출력결과

warrior

hp : 200

weapon : sword

archer

hp : 100

weapon : bow